Electrical and Computer Engineering Department

ENCS3390: Operating System Concepts

First Semester, 2024/2025

Project 1

Student Name: Mohammad Omar
Student ID: 1221332

Section: 1

Date: 25/11/202

CONTENTS

1. Computer Environment: 3

2. How you achieved the multiprocessing and multithreading requirements, i.e. The API and functions that you used. 4

2.1 Naive Approach: 4

2.1.2 API's and libraries used In the Program 4

2.2.2 Naive Workflow 4

2.1.3 Naive Result 5

2.2 Multiprocessing Approach: 6

2.2.1 API's and libraries used In the Program 6

2.2.2 Multiprocessing Workflow 8

2.2.3 Multiprocessing Result 9

2.3 Multithreading Approach: 10

2.2.1 API's and libraries used In the Program 10

2.2.2 Multithreading Workflow 12

2.2.3 Multithreading Result 13

3. Amdahl's LAw 14

4. performance of the three approaches 17

5. Comments on the Differences in Performance 17

6. conclusion 18

1. COMPUTER ENVIRONMENT:

Table 1 Computer Environment

Speed	2.30 GHz
Memory	16 GB
Programming Language	Unix C
IDE	Clion
Virtual Machine	Ubuntu VB

In this project, I developed and ran my code using **Ubuntu Linux**. I chose to work with **C programming language** due to its efficiency and performance, especially when it comes to implementing multiprocessing and multithreading. Ubuntu, as my operating system, provided a stable environment with excellent support for compiling and running C programs.

I used two commands to run the code: -

- **gcc main.c -lm:** this command is to perform compile in Unix.
- **./a.out:** This command is to execute the program (Run) un Unix

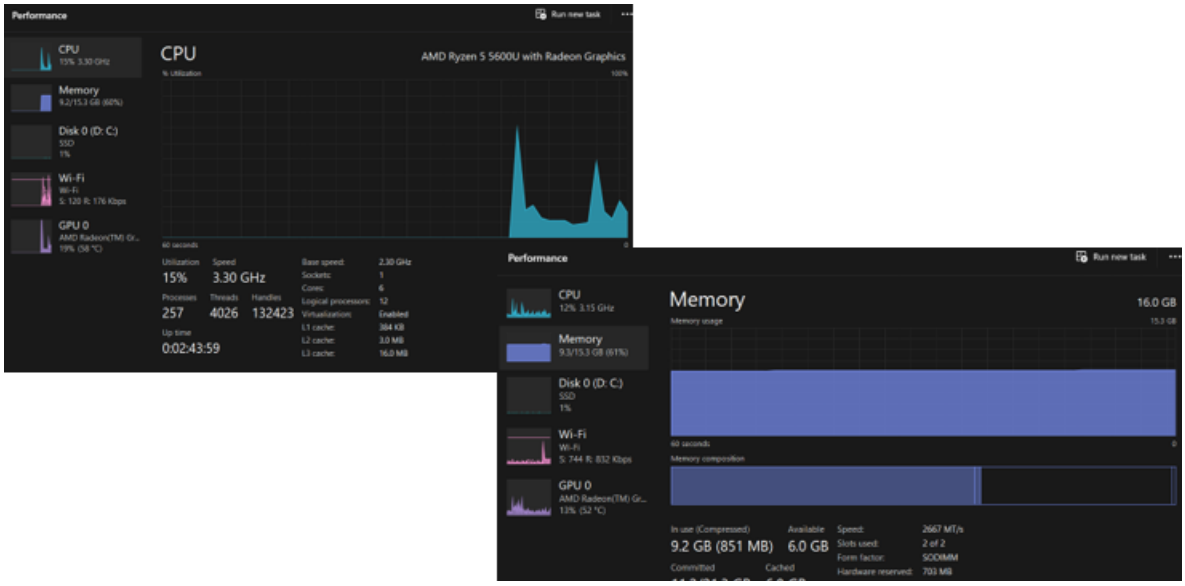

Figure 1 Computer Environment

2. HOW YOU ACHIEVED THE MULTIPROCESSING AND MULTITHREADING REQUIREMENTS, I.E. THE API AND FUNCTIONS THAT YOU USED.

2.1 NAIVE APPROACH:

This program demonstrates a simple, single-threaded approach to calculate the top 10 most repeated words in a large one-line text file. It does not utilize multiprocessing or multithreading. As there is no parallel processing, this approach is expected to perform the work slower than the multiprocessing and multithreading approach but provides a clear example of a basic implementation for frequency counting.

2.1.2 API'S AND LIBRARIES USED IN THE PROGRAM

In this approach, there are no special libraries invokes in the program, it's just an essential c programming library.

```
//naive Mohammad Omar 1221332
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/time.h>
```

Figure 2: Naive Approach library

2.2.2 NAIVE WORKFLOW

The naive approach follows these sequential steps:

1. **File Size Calculation**: First, the program determines the total number of words in the input file using function `fileSize()`, this function iterates through the file word by word, counting the total number of entries by increasing the counter.
2. **Data Loading**: after find the file size ,the words from the file are loaded into an array of struct (`bigArray`) that contain the word and integer value represent the frequency of it using the `loadFile()` function where each word sorted with initial frequency of 1.
3. **Word Frequency Calculation**: after loading the file into bigArrray, The `myWork()` function processes the `bigArray` to calculate word frequencies:

    - Each word is compared to existing entries in a smaller array (`smallArray`) that stores unique words and their frequencies that will printed later.
    - I declared an int variable (found) to represent a Boolean expression, if the word found then it will be 1 and if not it will be 0.
    - If a match is found, the frequency of the corresponding word is incremented.

- If no match is found, the word is added to the `smallArray` with an initial frequency of 1.

4. **Sorting and print the top 10 repeated words** : Once all word frequencies are calculated and stored in the array of struct (smallArray), the `qsort()` function is used to sort the `smallArray` in descending order of frequency, then the top 10 most frequent words are printed along with their frequencies in descending order.
5. Cleanup **and Time Calculation**: All dynamically allocated memory (`bigArray`, `smallArray`) is freed. Then the program calculates and displays the total execution time in minutes using the `gettimeofday()` function that build in <sys/time.h> library.

2.1.3 NAIVE RESULT

```
root@DESKTOP-SGCVK0I:/p2# gcc main.c -lm
root@DESKTOP-SGCVK0I:/p2# ./a.out
Top 10 most frequent words:
the: 1061396
of: 593677
and: 416629
one: 411764
in: 372201
a: 325873
to: 316376
zero: 264975
nine: 250430
two: 192644
Elapsed time: 10.37 minutes
root@DESKTOP-SGCVK0I:/p2#
```

Figure 3: Naive Approach Result

2.2 MULTIPROCESSING APPROACH:

This section details the implementation of the multiprocessing technique to efficiently calculate the top 10 most repeated words in a large one-line text file.

2.2.1 API'S AND LIBRARIES USED IN THE PROGRAM

The program employs many APIs and libraries to manage multiprocessing, shared memory, and synchronization effectively. The following table outlines the key APIs and their roles:

Table 2:Multiprocessing API's

API function	Functionality	Usage in the program
`fork()`	Creates a new child process.	Spawns multiple child processes to process chunks of data in parallel.
`mmap()`	Allocates a shared memory region accessible by all processes.	Creates shared memory (`shared_memory`) for storing the result array (`NewArray`).
`sem_init()`	Initializes a semaphore for synchronization.	Sets up the global semaphore `soso` to protect shared memory from race conditions.
`sem_wait()`	Locks the semaphore before accessing shared memory.	Ensures exclusive access to `NewArray` when updating shared memory.
`sem_post()`	Unlocks the semaphore after accessing shared memory.	Releases the lock after updating the shared memory.
`sem_destroy()`	Destroys the semaphore to free resources.	Cleans up the semaphore after the program finishes.

waitpid()	Waits for child processes to finish.	Ensures the parent process waits until all child processes complete before proceeding further.
qsort()	Provides quick sorting functionality.	Sorts the merged array of words in descending order by frequency.
exit()	Terminates a process.	Used to safely exit child processes and handle errors.
gettimeofday()	Measures elapsed time.	Records the start and end times of the program to calculate execution duration.
malloc()	Allocates dynamic memory.	Allocates memory for local arrays (`smallArray`) in child processes.
free()	Frees dynamically allocated memory.	Deallocates memory for `smallArray` after processing is complete.

To implement these API's, I called many library's: -

```
//multiprocessing Mohammad Omar 1221332
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/time.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/mman.h>
#include <semaphore.h>
```

Figure 4: Multiprocessing Library's

1 **<sys/time.h>:** Used for time operations which include **gettimeofday**() which is used in elapsed time calculations.
2 **<unistd.h>:** Provides an access to POSIX OS API's, including functions like **fork()** for creating child processes and **getpid()** to retrieve process IDs.

3 **<sys/mman.h>:** This library provides memory management functions, such as **mmap()** for creating shared memory between processes and **munmap()** for freeing the created shared memory.

4 **<semaphore.h>:** Provides functions to work with semaphores, such as **sem_init()** to initialize a semaphore, **sem_wait()** to acquire a semaphore, **sem_post()** to release it, and **sem_destroy()** to clean it up. These are essential for synchronizing access to shared resources in multiprocessing which avoid race conditions.

2.2.2 MULTIPROCESSING WORKFLOW

The multiprocessing approach follows these steps:

1. **Input and Setup**: The user enter the number of child processes to execute the program. Based on this input, the API **fork()** is called to create the required number of child processes. Each child operates independently, while the parent process should wait for their completion using **waitpid().**
2. **Dived the file data on the child processes**: The dataset (bigArray) is divided into smaller chunks (smallArray) based on the number of processes. while the file contains 17,005,207 words and there are for example 4 processes, each process will handle a chunk with a size:
   $chunck\ size = \frac{17{,}005{,}207}{4\ (\#\ of\ processes)}$.
3. **Processing in Child Processes**: Each child process calculates the frequency of words in its respective chunk using the **myWork()** function. This involves identifying unique words and incrementing their frequencies.
4. **Shared Memory Synchronization**:
   Shared memory (`sharedData`) is used to store and consolidate results from all processes. A semaphore (`sem_t soso`) synchronizes access to prevent race conditions:
   - **sem_wait()**: Ensures that only one process can modify shared memory at a time.
   - **sem_post()**: Releases the semaphore, allowing other processes to access the shared memory.
5. **Merging Results**: After all child processes complete, the results in the shared memory are merged using the `merge()` function. This function operates similarly to `myWork()`, consolidating word frequencies.
6. **Sorting and Display**: The consolidated word frequency array is sorted in descending order using `qsort()`,Then the top 10 most frequent words are printed.
7. **Cleanup**: after the program end use the memory allocated (malloced) elements, Dynamically they cleaned up. Shared memory are freed using `free()` and `munmap()`, the

semaphore is destroyed using `sem_destroy()`, the program calculates and prints the elapsed time.

2.2.3 MULTIPROCESSING RESULT

```
root@DESKTOP-SGCVK0I:/p2# gcc main.c -lm
root@DESKTOP-SGCVK0I:/p2# ./a.out
Enter the number of child processes you want: 2
Top 10 most frequent words:
the: 1061396
of: 593677
and: 416629
one: 411764
in: 372201
a: 325873
to: 316376
zero: 264975
nine: 250430
two: 192644
Elapsed time: 7.43 minutes
```

Figure 5: Multiprocessing using 2 child's output

```
root@DESKTOP-SGCVK0I:/p2# gcc main.c -lm
root@DESKTOP-SGCVK0I:/p2# ./a.out
Enter the number of child processes you want: 4
Top 10 most frequent words:
the: 1061396
of: 593677
and: 416629
one: 411764
in: 372201
a: 325873
to: 316376
zero: 264975
nine: 250430
two: 192644
Elapsed time: 5.06 minutes
```

Figure 6: Multiprocessing using 4 child's output

```
root@DESKTOP-SGCVK0I:/p2# gcc main.c -lm
root@DESKTOP-SGCVK0I:/p2# ./a.out
Enter the number of child processes you want: 6
Top 10 most frequent words:
the: 1061396
of: 593677
and: 416629
one: 411764
in: 372201
a: 325873
to: 316376
zero: 264975
nine: 250430
two: 192644
Elapsed time: 4.11 minutes
```

Figure 7: Multiprocessing using 6 child's output

```
root@DESKTOP-SGCVK0I:/p2# gcc main.c -lm
root@DESKTOP-SGCVK0I:/p2# ./a.out
Enter the number of child processes you want: 8
Top 10 most frequent words:
the: 1061396
of: 593677
and: 416629
one: 411764
in: 372201
a: 325873
to: 316376
zero: 264975
nine: 250430
two: 192644
Elapsed time: 3.50 minutes
```

Figure 8: Multiprocessing using 8 child's output

2.3 MULTITHREADING APPROACH:

This section explains the multithreaded implementation for calculating the top 10 most frequent words in a large text file. By dividing the loaded fata (stored in BigArray) among multiple threads.

2.2.1 API'S AND LIBRARIES USED IN THE PROGRAM

The multithreading implementation uses both standard C libraries and the **POSIX Threads (pthreads)** library. The table below summarizes the key functions and their purposes:

Table 3:Multiprocessing API's

API function	Functionality	Usage in the Program
`pthread_create()`	Creates a new thread to execute a specific function.	Launches threads to process portions of the input array concurrently.
`pthread_join()`	Waits for a thread to terminate.	Ensures all threads complete execution before proceeding to the next phase.
`pthread_mutex_init()`	Initializes a mutex lock.	Prepares the mutex (`soso`) for synchronizing access to the shared frequency array.
`pthread_mutex_lock()`	Acquires a mutex lock, blocking other threads.	Ensures exclusive access to the shared array (`sharedArray`) while merging local arrays.
`pthread_mutex_unlock()`	Releases a mutex lock.	Allows other threads to access the shared array after the current thread completes its updates.
`sem_destroy()`	Destroys the semaphore to free resources.	Cleans up the semaphore after the program finishes.

qsort()	Provides quick sorting functionality.	Sorts the shared frequency array in descending order of word frequency.
gettimeofday()	Measures elapsed time.	Records the start and end times of the program to calculate execution duration.
malloc()	Allocates dynamic memory.	Allocates memory for the input array (`bigArray`) and local frequency arrays for threads.
free()	Frees dynamically allocated memory.	Deallocates memory used by the input array and local arrays after processing.

To implement these API's, I called many library's: -

```
//multithreading Mohammad Hasan Omar
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <pthread.h>
#include <sys/time.h>
```

Figure 9: Multithreading Library's

1. **`<pthread.h>`**: Provides functions for creating and managing threads.
2. **`<sys/time.h>`**: Measures execution time for performance analysis.
3. **`<stdio.h>`, `<stdlib.h>`, `<string.h>`**: Offers standard I/O, memory management, and string manipulation utilities.

2.2.2 MULTITHREADING WORKFLOW

The multithreading approach follows these steps:

1. **Input Data Preparation**: The total number of words in the file is calculated using the `fileSize()` function, then words are loaded into a dynamically allocated array (`bigArray`) using the `loadFile()` function.

   Thread Initialization: at the first of the run the user is supposed to enter the number of threads to complete the work, then the loaded array (bigArray) will divided between the threads equally, that's mean each thread will take $\frac{17{,}005{,}207}{(\#\ of\ threads)}$ of words to do the work on them.

2. **Local Frequency Calculation**: Each thread independently calculates word frequencies for its chunk using the `threadJob()` function. A local array is used to store unique words and their frequencies for the thread's portion of the input array.
3. **Merging Results**: After processing, threads merge their local arrays into a shared frequency array (`sharedArray`) that is a normal array not like multiprocessing because the threads are already shared memory while the process need mmap() function to create shared memory. But as any shared memory there are need for synchronization using a mutex (`pthread_mutex_lock` and `pthread_mutex_unlock`) to avoid data races.
4. **Sorting and Output**: Once all threads have completed their work, the shared frequency array (SharedMemory) is sorted in descending order using `qsort()` and then the top 10 most frequent words are printed.
5. **Cleanup and Time Calculation**: All dynamically allocated memory is freed, and the mutex is destroyed. And The program calculates and displays the total execution time.

2.2.3 MULTITHREADING RESULT

```
root@DESKTOP-SGCVK0I:/p2# gcc main.c -lm
root@DESKTOP-SGCVK0I:/p2# ./a.out
Enter the number of threads you want: 2
Top 10 most frequent words:
the: 1061396
of: 593677
and: 416629
one: 411764
in: 372201
a: 325873
to: 316376
zero: 264975
nine: 250430
two: 192644
Elapsed time: 7.22 minutes
```

Figure 10: Multithreading using 2 child's output

```
root@DESKTOP-SGCVK0I:/p2# gcc main.c -lm
root@DESKTOP-SGCVK0I:/p2# ./a.out
Enter the number of threads you want: 4
Top 10 most frequent words:
the: 1061396
of: 593677
and: 416629
one: 411764
in: 372201
a: 325873
to: 316376
zero: 264975
nine: 250430
two: 192644
Elapsed time: 4.80 minutes
```

Figure 11: Multithreading using 4 child's output

```
root@DESKTOP-SGCVK0I:/p2# gcc main.c -lm
root@DESKTOP-SGCVK0I:/p2# ./a.out
Enter the number of threads you want: 6
Top 10 most frequent words:
the: 1061396
of: 593677
and: 416629
one: 411764
in: 372201
a: 325873
to: 316376
zero: 264975
nine: 250430
two: 192644
Elapsed time: 3.58 minutes
```

Figure 12: Multithreading using 6 child's output

```
root@DESKTOP-SGCVK0I:/p2# gcc main.c -lm
root@DESKTOP-SGCVK0I:/p2# ./a.out
Enter the number of threads you want: 8
Top 10 most frequent words:
the: 1061396
of: 593677
and: 416629
one: 411764
in: 372201
a: 325873
to: 316376
zero: 264975
nine: 250430
two: 192644
Elapsed time: 3.42 minutes
```

Figure 13: Multithreading using 8 child's output

3. AMDAHL'S LAW

The Amdahl's law is used to

$$speedup \leq \frac{1}{S + \frac{(1-S)}{N}}$$

- Identifies performance gains from adding additional cores to an application that has both serial and parallel components
- S is serial portion
- N processing cores

$$speedup = \frac{sequential\ time(Naive)}{parallel\ time(8\ childs)}$$

Naïve Time = 622.2 seconds

For multiprocessing: -

$$speedup = \frac{622.2}{210.0} = 2.962$$

$$s = \frac{\left| \frac{1}{speedup} - 1 \right|}{N - 1} = 0.0946 \approx 9.5\ \%$$

For multithreading: -

$$speedup = \frac{622.2}{205.2} = 3.032$$

$$s = \frac{\left| \frac{1}{speedup} - 1 \right|}{N - 1} = 0.09574 \approx 9.5\%$$

The serial part from the multiprocessing and multithreading is almost the same ≈ 9.5 %
Here we notice that the serial part is 9.5% and the parallel part is (100-9.5) % which equal to 90.5%
This vale is seeming to be realistic because the code work in serial in loading the file and another small jobs, while its working is the most in the parallel part (where the job is divided between Childs and thraeds).

# of Childes/ Approach	*Naive*	*Multiprocessing*	*Multithreading*
2 Childs	*10.37 min*	*7.34 min*	*7.22 min*
4 Childs		*5.06 min*	*4.80 min*
6 Childs		*4.11 min*	*3.58 min*
8 Childs		*3.50 min*	*3.42 min*

In seconds

	2 child's	4 child's	6 child's	8 child's
Multiprocessing	445.8	303.6	246.6	210.0
Multithreading	433.2	288.0	214.8	205.2

Naïve Time = 622.2 seconds.

If we take a ratio by divide the time of the naïve on the time of the multi-processing and multithreading child it will give: -

$$speedup = \frac{622.2}{parallel\ time(n\ childs)}$$

from the table, as the number of child processes or threads increases, the execution time decreases. The optimal number of child processes or threads is the point where adding more does not significantly improve performance or where the system's limitations are reached.

Thus, **4 child processes/threads** appear to be the optimal number for this dataset and workload configuration, provided the system resources can support this level of parallelism without causing overhead or contention.

	2 child's	4 child's	6 child's	8 child's
Multiprocessing	1.39	2.049	2.523	2.961
Multithreading	1.436	2.16	2.89	3.032

According to Amdahl's law results, the optimal number of children or threads is 6

4. PERFORMANCE OF THE THREE APPROACHES

Approach	Memory Usage	Concurrency Level	Scalability	Implementation Complexity
Naive	Low	None	Limited	Simple
Multiprocessing	High	Process-based	High (better CPU utilization)	Complex
Multithreading	Medium	Thread-based	Moderate (depends on CPU cores)	Moderate

5. COMMENTS ON THE DIFFERENCES IN PERFORMANCE

- The naive approach to processing data is single-threaded, which is slow for large datasets but has minimal memory usage. It is suitable for small datasets or simple environments.
- The multithreading approach splits the work among threads, improving performance, but is limited by the Global Interpreter Lock in Python.
- Multiprocessing is the most efficient for large datasets, using independent processes for each core, but has higher memory usage and complexity with inter-process communication overhead.

6. CONCLUSION

In this project, I applied three methods for handling massive amounts of data are compared in this report: naïve, multithreading, and multiprocessing. Every technique has unique benefits and disadvantages.

- Naive method is easy to set up and memory-efficient, but slowest performance due to lack of scalability and concurrency so only appropriate for settings where performance is not as important as simplicity or for tiny datasets.
- Multi-Threading method maintains a balance between memory and performance. effective with datasets that are relatively large, but restricted by possible thread management costs and CPU core availability. The synchronization of shared memory adds complexity.
- Multi- processing method provides optimal performance and scalability for big datasets. uses all of the CPU cores in the other hand, its excessive memory consumption and complexity because of the need for synchronization and inter-process communication.
- According to Amdahl's law results, the optimal number of children or threads is 6

According to the performance comparison, the multiprocessing method provides the least amount of execution.

According to Amdahl's law results, the optimal number of children or threads is 6

$$Multithreading > Multitasking > Naive$$